

Nameless parameters and unutterable specializations

Document number: P0736R0
Date: 2018-02-12
Project: ISO/IEC JTC 1/SC 22/WG 21/C++
Working Group: Evolution
Revises: None
Reply-to: Hubert S.K. Tong <hubert.reinterpretcast@gmail.com>

Issue/Background

When an expression is part of a signature for a function template, whether expressions are equivalent for the purposes of resolving declarations of the same entity to each other is defined by [temp.over.link]. This definition in turn relies on the form of the expression in the style of the one-definition rule ([basic.def.odr]).

Unfortunately, it is not specified what form an expression takes when template parameters are substituted with their corresponding arguments. Indeed, with the addition of *requires-clauses*, substitution might not occur in certain subexpressions. This frustrates the ability to form redeclarations in contexts where certain template parameters have no name, such as when an enclosing template is explicitly specialized. Redeclarations in such contexts also obfuscate the relationship between declarations expressed in terms of the primary template and declarations expressed with reference to specializations.

Current implementation behaviour

Even without the addition of Concepts, it is possible to encounter the problem of lacking a suitable way to form equivalent corresponding expressions in redeclarations.

For example, given:

```
template <unsigned N>
struct TA {
    template <unsigned M>
    void f(unsigned (*)[N + M]);
};
```

An attempt to produce an explicit specialization of `f` for `TA<0>` might yield:

```
template <>
template <unsigned M>
void TA<0>::f(unsigned (*)[0u + M]);
```

Indeed, it happens to work with both GCC and Clang. Unfortunately, not all literal types can necessarily have their values expressed as integer literals (`short` comes to mind). In any case, it only happens to work and may stop working at any time. GCC (at least as late as version 7.2 from August 2017) would have accepted `0` in place of `0u` despite the type mismatch (so the `short` conundrum was somewhat sidestepped), but no longer does so.

Now with the addition of Concepts, an additional complication appears.

Given:

```
template <typename T>
struct TB {
    template <typename U>
    void f() requires U::value && T::value;
};
```

A similar approach to forming the explicit specialization of `f` for `TB<int>` would create the unutterable `int::value`. I am not aware of any version of the Concepts implementation that does not reject the utterance of `int::value`.

Possible solutions

It is the hope of the author that either a solution is adopted to ease the formation of related declarations or the effective inability to produce truly equivalent expressions in said contexts is made clear through examples to be incorporated into the Standard.

In the pursuit of a solution, syntax to allow binding of a name for specialized template arguments of the primary template could be explored.

In the context of the above cases, such syntax may look like this:

```
template <>
template <unsigned M>
void TA<N: 0>::f(unsigned (*)[N + M]);

template <>
template <typename U>
void TB<T: int>::f() requires U::value && T::value;
```

Notice that in the latter case, `T` would need to be treated as dependent.

Such a binding could also allow for partial specializations to have constraints expressed in terms of the parameters to the primary template; however, an extra complication is needed to introduce the names before their use:

```
template <typename T, typename U>
struct A;

[T, U] template <typename TT, typename UU>
requires C<T, U>
struct A<T: TT *, U: UU> { };

[T, U] template <typename TT, typename UU>
requires C<T, U> && C1<T>
struct A<T: TT, U: UU *> { };
```

Notice that without the binding, the intended subsumption of `C<T, U>` in the first specialization by the *requires-clause* of the second would be muddled.

Thus we speak the unutterable and name the nameless.